QUIZ PERTEMUAN 1

PEMROGRAMAN BERORIENTASI OBJEK

Oleh:

Zeyan Fahluzi Alhariz

250660221033

SOAL

1. Konsep Class & Object

   Jelaskan perbedaan Class dan object dengan kalimatmu sendiri, lalu berikan satu contoh dari sistem perpustakaan (sebutkan mana yang menjadi class dan mana yang menjadi object).

Jawaban:

- Perbedaan Class dan Object:
- Class adalah cetakan, cetak biru (*blueprint*), atau rintisan yang mendefinisikan atribut (*data*) dan perilaku (*method*) dasar yang akan dimiliki oleh suatu objek.
- Object adalah hasil wujud nyata (instansiasi) yang dibuat berdasarkan cetakan *class* tersebut, di mana objek memiliki nilai atau data spesifiknya sendiri.
- Contoh dari Sistem Informasi Perpustakaan:

- Class: `Buku` (merupakan cetakan yang menentukan bahwa setiap buku akan memiliki judul, penulis, dan status peminjaman).
- Object: `buku1 = Buku("Laskar Pelangi", "Andrea Hirata", "Tersedia")` (merupakan wujud fisik/objek nyata dari buku tertentu di dalam sistem).

2. Memahami self

A. Apa fungsi self di dalam method python?
B. Apa yang terjadi jika parameter self dihilangkan dari sebuah instance method,lalu method tersebut dipanggil lewat object? Tuliskan pesan error yang muncul.

Jawaban:

a. Fungsi `self` di dalam method Python: `self` berfungsi sebagai referensi yang menunjuk ke objek itu sendiri (instance spesifik). Dengan adanya `self`, sebuah *method* dapat mengakses atau mengubah atribut dan *method* lain milik objek yang sedang memanggilnya.

b. Dampak dan Pesan Error jika `self` dihilangkan: Jika parameter `self` dihilangkan dari sebuah *instance method* dan *method* tersebut dipanggil melalui objek, Python akan menghasilkan `TypeError`. Hal ini terjadi karena saat memanggil *method* lewat objek, Python secara otomatis mengirimkan objek tersebut sebagai argumen pertama, tetapi definisi *method* tidak menyediakan parameter untuk menerimanya.

• pesan error yang muncul:

TypeError: nama_method() takes 0 positional arguments but 1 was given

3. Prediksi output

```python
class Mahasiswa:
    def __init__(self, nama, nim):
        self.nama = nama
        self.nim = nim

    def sapa(self):
        print(f"Halo, saya {self.nama} ({self.nim})")


m1 = Mahasiswa("Budi", "SI-101")
m2 = Mahasiswa("Ani", "SI-102")

m1.sapa()
m2.sapa()
```

Tuliskan output yang muncul saat kode di jalankan, dan jelaskan mengapa m1 dan m2 menghasilkan output yang berbeda padahal keduanya dibuat dari class yang sama.

Jawaban:

Outpun yang muncul: Halo, saya Budi (SI-101) •Halo, saya Ani (SI-102)

Mengapa output berbeda

Meskipun `m1` dan `m2` dibuat dari *class* yang sama (`Mahasiswa`), keduanya merupakan dua *object* (instance) yang berbeda yang disimpan pada lokasi memori terpisah.

1. Atribut Instance Independen: Saat objek `m1` diinstansiasi, konstruktor `__init__` menyimpan nilai `nama = "Budi"` dan `nim = "SI-101"` khusus untuk instance `m1`. Begitu pula saat `m2` dibuat, `__init__` menyimpan nilai `nama = "Ani"` dan `nim = "SI-102"` khusus untuk instance `m2`.
2. Peran `self`: Saat `m1.sapa()` dipanggil, parameter `self` di dalam method `sapa()` merujuk ke instance `m1`, sehingga yang dicetak adalah atribut milik `m1`. Sebaliknya, saat `m2.sapa()` dipanggil, `self` merujuk ke instance `m2`.

4.Cari dan perbaiki error

Kode berikut memiliki 2 kesalahan. Temukan, sebutkan, dan perbaiki

```
class Buku:
    def __init__(self, judul):
        self.judul = judul

    def info():
        print(f"Judul: {judul}")


b = Buku("Pemrograman Python")
b.info()
```

Kesalahan 1: Masalah: Parameter `self` tidak dimasukkan ke dalam definisi method `info()` (ditulis `def info():`).

Dampak: Saat dipanggil menggunakan `b.info()`, Python akan mengirimkan objek `b` secara otomatis sebagai argumen pertama sehingga menyebabkan `TypeError`.

Kesalahan 2: Masalah: Di dalam method `info()`, variabel `judul` diakses langsung tanpa awalan `self.` (ditulis `{judul}`).

Dampak: Python akan mencari variabel lokal bernama `judul` yang tidak ada di dalam fungsi tersebut, sehingga menyebabkan `NameError`. Untuk mengakses atribut instance objek, harus menggunakan `self.judul`.

Kode yang sudah diperbaiki:

```
class Buku:
    def __init__(self, judul):
        self.judul = judul

    def info(self):
        print(f"Judul: {self.judul}")


b = Buku("Pemrograman Python")
b.info()
```

5.Construktor & Class Attribute

Perhatikan kode berikut:

```
class Produk:
    total_produk = 0

    def __init__(self, nama):
        self.nama = nama
        Produk.total_produk += 1


a = Produk("Laptop")
b = Produk("Mouse")
c = Produk("Keyboard")

print(Produk.total_produk)
print(a.nama)
```

a.Kapan method __ init __() dipanggil? Apa fungsinya?

b. Tuliskan output kedua perintah print di atas, dan jelaskan perbedaan total_produk (class attribute) dengan nama (instance attribute).

Jawaban:

a. Waktu Pemanggilan & Fungsi `__init__()`

- Kapan dipanggil: Method `__init__()` dipanggil secara otomatis oleh Python pada saat sebuah *object* (instance) baru dari *class* dibuat (diinstansiasi).
- Fungsinya: Berfungsi sebagai *constructor* untuk menginisialisasi nilai awal dari *attribute-attribute* milik objek yang baru saja dibuat tersebut.

b. Output & perbedaan attribute

Output:

3

Laptop

Perbedaan `total_produk` (Class Attribute) dan `nama` (Instance Attribute):

- `total_produk` (Class Attribute):
- Scope: Dimiliki oleh *class* itu sendiri dan bernilai sama untuk seluruh *object*.

- Sifat: Nilainya terbagi (shared) ke seluruh instance. Ketika nilainya bertambah (`Produk.total_produk += 1`), perubahan ini berlaku untuk seluruh class dan semua objeknya.
- `nama` (Instance Attribute):
- Scope: Dimiliki secara khusus oleh masing-masing *object* (instance) individu melalui `self.nama`.
- Sifat: Nilainya bersifat independen (berbeda-beda) untuk tiap objek, bergantung pada nilai yang dikirimkan saat pembuatan objek (misal: objek `a` memiliki nama `"Laptop"`, sedangkan `b` memiliki nama `"Mouse"`).